STRIPE PROJECT

1. OLTP Data System

Annexes

1.D. OLTP Disaster Recovery Strategy

> *Nicolas Pichon - AIA RNCP 38777 / BC02 / OLTP Data System - Annexe D / v30 - 2026/10/13.*

1.D.1. Contexte

Ce document annexe expose les stratégies de réplication, de basculement et de reprise après sinistre du système *OLTP*. La procédure de recouvrement est exigée dans le cas d'étude [R0] via l'exigence d'affaire *BR1* (*"disaster recovery"*, *"real-time data synchronization"*).

On se place dans un contexte où le *SGBD OLTP* est supporté par *PostgreSQL*.

1.D.2. Principes

Le système *OLTP* supporte l'écriture de données de criticité très différentes :

- Les données d'*enjeu critique*, financier ou contractuel :
  - `Transaction`, `Subscription`, `Refund`, `Chargeback`, `FraudScore`, `FraudDecision`, `FraudScoringLog` ;
- Les journaux de sécurité et conformité :
  - `DataAccessLog`, `ChangeAuditLog`, `SecurityIncident`, `ConsentRecord`, `DataSubjectRequest`, `FraudScoringLog`, `FraudReview`) ;
- Les liste de références :
  -
    - `Country`, `Currency`,s statuts, etc.

Chacune de ces familles des données a des exigences de continuité propres :

Criticité	RPO (Recovery Point Objective)	RTO (Recovery Time Objective)
Enjeu critique	~0 (aucune transaction confirmée ne doit être perdue)	Quelques secondes à ~1 minute
Journaux	Jusqu'à ~1 minute (écriture déjà asynchrone par conception)	Quelques minutes
Référentiels	Quelques minutes	Quelques minutes (lecture seule tolérable en repli)

> La disparité des exigences de continuité justifie une architecture à *réplication différenciée* plutôt qu'une politique unique pour toute la base.

1.D.3. Réplication

1.D.3.1. Réplication physique en streaming

La réplication physique en flux est assurée par le mécanisme de *streaming replication* de *PostgreSQL*. Au moins deux répliques sont mises en place :

- 1 réplique *synchrone dans la même région* (zone de disponibilité différente) :
  - Le commit d'une transaction sur `Transaction` n'est confirmé au client qu'une fois répliqué sur ce noeud (`synchronous_commit = on`, `synchronous_standby_names` ciblant ce noeud) ;
  - Ce mécanisme assure un 'RPO' négligeable sur l'écriture des données lors de la transaction financière (chemin d'exécution critique) ;

- 1 ou plusieurs répliques *asynchrones* supplémentaires, dans une région géographique distincte :
  - L'objectif est d'absorber la perte d'une région entière (dans le cadre d'une reprise après sinistre) sans pénaliser la latence d'écriture des données d'enjeu critique.

1.D.3.2. CDC (Change Data Capture)

CDC via réplication logique (`pgoutput` / Debezium) en parallèle de la réplication physique : c'est le mécanisme modélisé dans `ChangeAuditLog` et documenté comme alimentant OLAP/ NoSQL de façon asynchrone, hors chemin transactionnel.

La réplication physique assure la continuité de service. Le *CDC* logique assure la synchronisation vers les systèmes analytiques (couvre *BR4*/*TR4*).

1.D.4. Bascule (failover)

1.D.4.0. Introduction

Dans un écosystème de réplication, il y a plusieurs plusieurs copies de la base :

- une **primaire** (la seule base seul qui accepte les écritures),
- des **répliques** (qui reçoivent en continu les états de la copie primaire).

Le but du processus de *bascule* (*failover*) est de promouvoir une *réplique* en nouveau *primaire* en cas de panne du *primaire*, et à rediriger les applications vers elle, le plus vite possible et sans perdre de transactions.

Le *failover* est un processus en quatre étapes :

1. *Détection* : constater/confirmer que le primaire est en panne ;
2. *Élection* : choisir la réplique qui remplace le primaire en panne ;
3. *Promotion* : transformer techniquement cette réplique en primaire ;
4. *Redirection* : faire en sorte que (1) les applications se connectent au nouveau primaire, (2) les anciennes répliques suivent le nouveau.

Ce que *PostgreSQL* propose *nativement* :

Étape	Natif PostgreSQL	Solution alternative
Réplication en continu du primaire vers les répliques (streaming replication)	Oui	-
Réplication synchrone (commit confirmé après l'actualisation de la réplique)	Oui	-
Promotion d'une réplique en primaire	Oui (manuelle)	-
Détection de la panne du primaire	Non	-
Élection automatique de la meilleure réplique	Non	-
Redirection des applications	Non	-
Remise en ordre des autres répliques et de l'ancien primaire	Non (orchestration manquante)	-

La détection *et la confirmation* de panne, l'élection du nouveau primaire, la redirection des applications, l'orchestration des répliques nécessitent une source de vérité unique et fiable :
...

Fenêtre de bascule

La durée d'une bascule doit rester en dessous de 30 secondes. C'est la durée pendant laquelle le service d'écriture est interrompu : de l'instant où le primaire cesse de répondre jusqu'à celui où les applications peuvent de nouveau écrire sur le nouveau primaire. La fenêtre proposée est cohérente avec le *RTO* des données critique donné en §1.D.2.

Limites de la proposition

La proposition suivante n'intègre pas la supervision (*monitoring*) des pannes et états du système de *failover*.

Détection et bascule automatiques

La détection et la bascule automatiques sont assurées par un gestionnaire de haute disponibilité [...]
élection d'un nouveau primaire parmi les répliques synchrones en cas de perte du noeud primaire, sans intervention manuelle.

[...]

Solutions proposées : *Patroni+etcd* ou un équivalent managé par le fournisseur cloud (*RDS Multi-AZ*, *Cloud SQL*, *Aurora*).

Patroni est un outil open source qui permet d'orchestrer la première partie de la procedure de failover (détection, élection, promotion).
Un agent Patroni tourne **à côté de chaque instance PostgreSQL** et la pilote :

- *Sur le primaire* :
  - renouvelle en permanence le verrou de leader.
  - auto-protection : rétrograde le primaire s'il n'arrive plus à joindre *etcd*. C'est la parade au *split-brain* : un primaire isolé sait qu'il a peut-être perdu son statut et se met en sécurité.
- *Sur les répliques* :
  - surveille le verrou. S'il expire (le primaire ne répond plus), les répliques candidates entrent en compétition ; celle qui obtient le verrou gagne (avec un critère de fraîcheur : on n'élit pas une réplique en retard sur les données).
  - réaligne les autres répliques sur le nouveau primaire,
  - réintégre éventuellement l'ancien primaire comme réplique.

etcd est un magasin clé-valeur, open-source, distribué, fortement cohérent. Son rôle dans le système de *failover* est d'héberger les informations suivantes :

- *Le verrou de leader* : une clé du type « le primaire actuel, c'est le nœud A », que le primaire doit renouveler régulièrement (un bail, avec une durée de vie de quelques secondes).
- *L'état partagé* : configuration du cluster, etc.

Protection contre le blocage de *Patroni* (*watchdog*)

Si le processus *Patroni* se fige alors que *PostgreSQL* continue de tourner, l'ancien primaire peut rester actif après la promotion d'une autre réplique. Un *watchdog* matériel ou logiciel force l'arrêt du nœud dans ce cas.

Supervision et alertes

Une bascule peut avoir lieu sans que personne ne le sache. Il faut surveiller l'API et les métriques de *Patroni*, l'état d'*etcd*, le retard de réplication et les événements de bascule.

Sauvegardes

Si la procédure de *rewind* native échoue ou n'est pas possible, il faut reconstruire le nœud depuis une sauvegarde.

Proxy de connexion

Pour permettre à la procédure de failover de rediriger les applications vers le nouveau primaire sans qu'elles s'en aperçoivent, un proxy de connexion doit être inséré devant le cluster, de telle sorte que les applications ne référencent jamais directement un noeud physique.

Le rôle du proxy de connexion est, d'une part, d'exposer une adresse unique aux applications, d'autre part, d'interroger régulièrement l'API REST *Patroni* de chaque nœud pour identifier le primaire courant. Le proxy renvoie le trafic d'écriture au nœud primaire, et seulement au noeud primaire. Après une bascule, le nouveau primaire s'identifie, l'ancien ne répond plus, et le proxy redirige tout seul le trafic en quelques secondes.

Solutions de *proxy*

Solutions proposées : *HAProxy*, ou un service managé (*RDS Proxy*, *Cloud SQL Auth Proxy*, etc.).

HAProxy est un répartiteur de charge et un proxy réseau générique, open-source, et très répandu. Il reçoit des connexions sur une adresse fixe et les redirige vers un ou plusieurs serveurs en aval, en vérifiant régulièrement lesquels sont en bonne santé.

HAProxy travaille dans la couche de *transport réseau (TCP)* : il ne comprend pas le protocole *PostgreSQL* et transfère simplement les octets vers le bon serveur.

Pooling de connexions

Un déploiement du système *OLTP* à grande échelle implique la capacité à gérer des milliers de connexions clientes concurrentes en chaque noeud. Dans le cas d'un système basé sur *PostgreSQL*, cette exigence doit être couverte par une couche logicielle externe, dite de *pooling de connexions*, car *PostgreSQL* n'a pas nativement la capacité à absorber autant de connexions simultanées.

> *PostgreSQL* gère les connexions clientes concurrentes en dédiant un *processus système* à chaque connexion. Etant donné le coût d'un *processus système* en terme de ressource mémoire/CPU (comparé à un thread léger, par exemple), le serveur *PostgreSQL* s'écroulerait bien avant d'avoir absorber un millier de connexions (le nombre maximum de connexions que le serveur accepte de traiter simultanément est de 100 par défaut).

Solutions de *pooling*

Solutions proposées : *PgBouncer*, ou un service managé (généralement, le même que celui qui gère déjà le proxy de connexion : *RDS Proxy*, *Cloud SQL Auth Proxy*, etc.).

PgBouncer est un *pooler de connexions PostgreSQL*, open source, très léger. Il permet de gérer des milliers de connexions clientes concurrentes sans multiplier les processus *PostgreSQL* en exploitant le fait que seul un petit nombre de ces connexions sont actives au meme moment et en ne présentant à *PostgreSQL* qu'une dizaines de connexions réelles.

Répliques asynchrones inter-régions

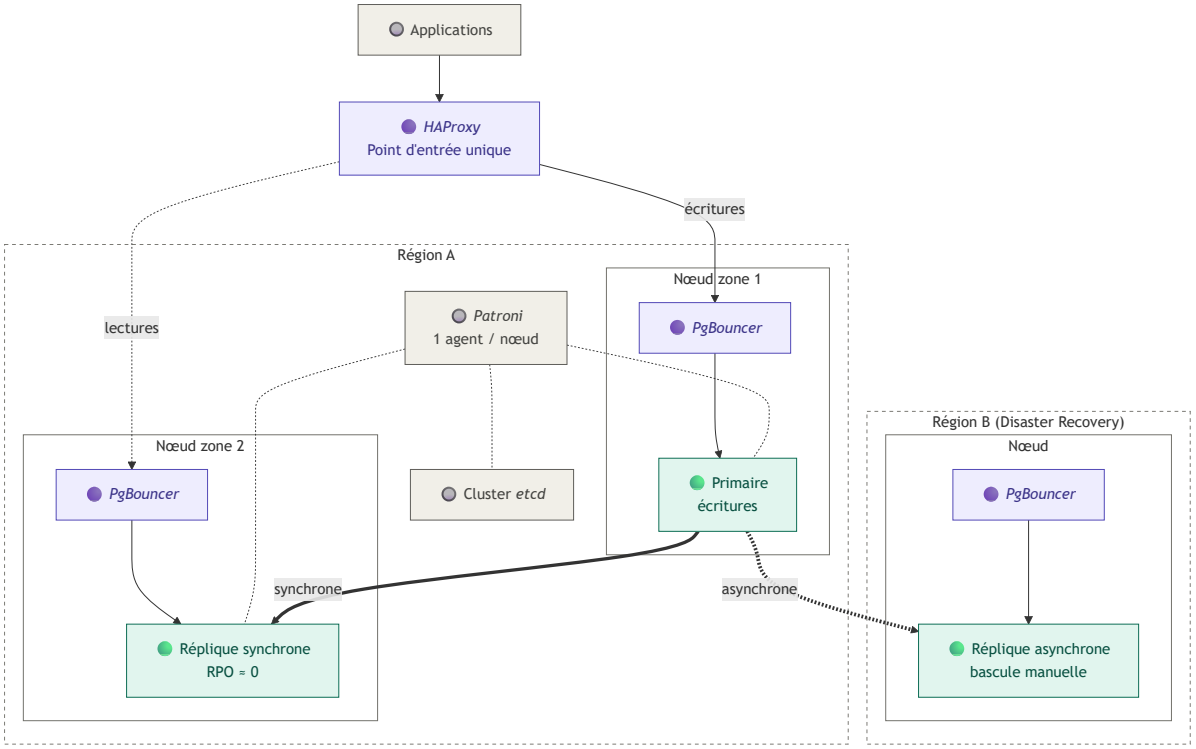
Les répliques *asynchrones inter-régions* ne participent pas à l'élection automatique (car la latence de réseau inter-régions est trop variable pour fiabiliser un quorum). Elles servent en cas de reprise après sinistre. Le tableau ci-dessous récapitule les différences en les deux types de répliques.

Failover (A) vs Disaster Recovery (B)

	Failover (région A)	**Disaster Recovery (région B)**
Déclencheur	Panne d'un nœud ou d'une zone	Perte de la région entière
Bascule	Automatique (*Patroni*)	Manuelle (décidée par un humain)
Réplique concernée	Synchrone	Asynchrone
Perte de données possible	Aucune (RPO ≈ 0)	Quelques secondes (RPO > 0)

> Les répliques asynchrones montrent toujours un léger retard de synchronisation par nature. La promotion d'une réplique asynchrone s'accompagne donc d'une perte probable des dernières transactions non encore répliquées (contrairement à se que présente le tableau 1.D.2).

... (diag)



> *Légende* :
>
> - : *les applications et le plan de contrôle (Patroni, etcd)*
> - : *les instances PostgreSQL (primaire et répliques)*
> - : *les proxys (HAProxy et PgBouncer)*
> - `---->` : Trafic applicatif
> - `-.->` : Supervision (bail, état)
> - `===>` : Réplication Synchrone
> - `=.=>` : Réplication Asynchrone

1.D.5. Reprise après sinistre (disaster recovery)

1.D.5.1. Sauvegardes physiques continues

Les sauvegardes physiques continues permettent de restaurer l'ensemble de la base dans l'état où elle à un point donné de la fenêtre de rétention.

> Sous *PostgreSQL*, les sauvegardes physiques continues sont assurées par deux processus complémentaires :
>
> - l'archivage continu du journal des modifications via le journal natif *WAL* (*Write Ahead Log* - le journal d'écriture anticipée) ;
> - un snapshot hebdomadaire complet de la base

> La gestion des sauvegardes et des restaurations sous *PostgreSQL* peut être assurée par *pgBackRest* ou par un équivalent managé. *pgBackRest* est une extension open source de *PostgreSQL* qui se charge des points suivants :

```
- recevoir & archiver les segments *WAL* ;
- réaliser les sauvegardes de base (complètes, différentielles, incrémentales) ;
- purger en fonction de la politique de rétention ;
- vérifier l'intégrité des sauvegardes (sommes de contrôle), compresser, chiffrer ;
- envoyer les sauvegardes vers différentes cibles (disque, S3, stockage objet cloud) ;
- orchestrer la restauration.
```

> Fenêtre de rétention recommandée pour les sauvegardes continues : 35 jours, conformément aux exigences de délai RGPD/PCI-DSS (cf.) `DataSubjectRequest.due_at`).

1.D.5.1. Bascule inter-région

En cas de perte totale de la région primaire, la bascule inter-région est déclenchée manuellement vers la réplique asynchrone distante, plutôt qu'automatisé, pour éviter un failover inter-région intempestif sur un simple pic de latence réseau. Le processus est documenté et testé régulièrement (exercice de bascule programmé, a minima trimestriel).

1.D.5.1. Vérifications d'intégrité après bascule

Un écart entre les valeurs des données précalculées (ex: `Transaction.fee_amount`) et leurs valeurs recalculées après restauration signale une réplication incomplète.

1.D.5.1. Trace de la perte des tables des journaux après bascule

Toute purge ou incohérence détectée sur `DataAccessLog` / `ChangeAuditLog` après un incident de bascule doit elle-même être tracée dans `SecurityIncident` , la table conçue pour capturer les incidents est aussi celle qui documente les incidents touchant l'infrastructure elle-même.

Couverture des exigences

- La stratégie proposée couvre les exigences suivantes [R0] :
  - BR1 (*"disaster recovery"*, *"real-time data synchronization"*),
  - TR3 (haute disponibilité). Reste hors périmètre de ce document, à traiter séparément : le choix précis du fournisseur/outillage managé (RDS Multi-AZ, Cloud SQL, Aurora...), qui est une décision d'infrastructure et non de modélisation de données.

Références

Ressources

- [R0] Stripe Business Case